

文献阅读笔记

一、vLLM (PagedAttention)笔记

vLLM: *Efficient Memory Management for Large Language Model Serving with PagedAttention* (SOSP 2023)

问题: 大模型推理时, KV Cache 需要占用大量空间。以前的系统要求 KV Cache 存在连续的内存里, 因为不知道模型最终会输出多长, 系统不得不按最大可能长度预分配显存, 这导致了非常严重的显存碎片和浪费, 大概有 60%-80%的显存会被白白浪费, 所以能同时处理的请求数非常少。

解决方案: 文献作者借鉴了操作系统里的“虚拟内存分页”思想, 提出了 PagedAttention。我的理解是, 这就是不再要求 KV Cache 连续存放, 而是把显存切成一个个固定大小的“块”, 用一张“页表”来记录逻辑上和物理上的对应关系, 用多少分配多少, 按需分配。

二、SGLang (RadixAttention) 笔记

SGLang: *Efficient Execution of Structured Language Model Programs* (NeurIPS 2024)

问题: 大模型在多轮对话、少样本提示或者 Agent 情况下会被多次调用。这些调用往往共享很长的前缀。像 vLLM 等传统系统, 一个请求结束后 KV Cache 就被丢弃了, 下个请求即使前缀相同也要重新算 Prefill, 重复计算非常浪费。

解决方案: 文献作者提出了 RadixAttention。这个的核心是用一棵“基数树”来管理所有请求的 KV Cache。新请求进来时, 先去树里找最长公共前缀。如果命中, 直接复用以前算过的 KV Cache, 跳过这部分计算。树上的节点还有 LRU 淘汰策略, 显存不够时自动删掉最久没用的叶子节点。

三、两者的联系与差异

联系: SGLang 的 RadixAttention 兼容并吸收了 vLLM 的 PagedAttention 的分页管理思想, 两者都在试图解决显存限制这个问题。

差异: vLLM 解决的是“单次请求内, 显存怎么高效分配”的问题, 避免了浪费, 而 SGLang 解决的是“跨请求、多次调用之间, 显存怎么高效复用”的问题, 避免了重复计算。